

NETWORK AND SYSTEM ADMINISTRATION

Lecture Notes

Chapter 1: Introduction

1 Network Administration Definition

Network administration aims to manage, monitor, maintain, secure, and service an organization's network. However, the specific tasks and procedures may vary depending on the size and type of an organization

2 Who is a network administrator?

A network administrator is an IT professional who ensures that an organization's computer networks (groups of computers sharing information) are operating to meet its needs. In addition, they can maintain and troubleshoot computer networks and employ security principles to keep networks secure.

3 What does a network administrator do?

Network administration primarily consists of, but isn't limited to, network monitoring, network management, and maintaining network quality and security.

Network monitoring is essential to monitor unusual traffic patterns, the health of the network infrastructure, and devices connected to the network. It helps detect abnormal activity, network issues, or excessive bandwidth consumption early on and takes preventative and remedial actions to uphold network quality and security.

Network management encompasses multiple administrative functions, including planning, implementation, and configuration. It involves:

1. Installing, monitoring, troubleshooting, and upgrading network infrastructure, including both hardware and software components
2. replanning the network based on changing organizational requirements
3. implementing optimization techniques to improve network efficiency and utilization
4. assessing the network for weaknesses
5. evaluating quality and capacity to increase or decrease network capacity and managing resource wastage
6. managing and granting network access to users and endpoint devices

Network security employs various techniques to ensure a network is secure. For example, it uses multiple tools such as firewalls, intrusion detection or prevention systems, and anti-malware software to prevent or detect malicious activity in the network. Not only that, but also applying security patches and updating the firmware of the networking infrastructure, such as routers, hubs, switches, and firewalls

4 Network administration key areas

Networks administration consists of 5 key areas:

1. **Fault management:** Monitors the network infrastructure to identify and address issues potentially affecting the network. It uses standard protocols such as Simple Network Management Protocol (SNMP) to monitor network infrastructure.
2. **Configuration management:** Tracks configuration and related changes of network components, including switches, firewalls, hubs, and routers. As unplanned changes can affect the network drastically and potentially cause downtime, it's essential to streamline, track, and manage configuration changes.
3. **Account management:** Tracks network utilization to bill and estimate the usage of various departments of an organization. In smaller organizations, billing may be irrelevant. However, monitoring utilization helps spot specific trends and inefficiencies.
4. **Performance management:** Focuses on maintaining service levels needed for efficient operations. It collects various metrics and analytical data to assess network performance, including response times, packet loss, and link utilization.
5. **Security management:** aims to ensure that only authorized activity and authenticated devices and users can access the network. It employs several disciplines, including threat management, intrusion detection, and firewall management. It also collects and analyzes relevant network information to detect and block malicious or suspicious activity.

5 System administration vs. network administration

Some organizations might use system administrator and network administrator interchangeably, and there are many overlapping responsibilities. But there are technically different. System administration focuses on servers and computer systems. Network administrators work more specifically with network-related tasks and equipment, like setting up routing and IP addresses and maintaining Local Area Networks (LAN). If you are in a smaller organization, these responsibilities might be folded into

one role, while larger organizations tend to differentiate them. However, since systems and networks are often intertwined, it is not rare to see job descriptions that require knowledge of both.

6 Five Things to be Considered in Designing a Network

When designing a new computer network, whether for five or 500 people, it is essential to balance the needs and desires of those who will use the network with the budget of those who will pay for its implementation. However, there are no options for implementing things that are mandatory for health network operations which are as follows:

1. **Connectivity and Security:** Network connectivity today means more than Ethernet cables and wireless access points. People today are more connected while mobile than ever before; many want access to company email and data while out of the office. Balancing those needs while maintaining security is a challenge that must be addressed in any network's design phase. This includes where data is stored, in-house or offsite, with cloud-based solutions, what types of information should be accessible, who should be able to access it, and which types of devices should be included. In addition, firewalls and access servers need to be secure without slowing down operations.
2. **Redundancy and Backing Up:** Redundancy means having backup devices in place for any mission-critical components in the network. Even small organizations should consider using two servers. Two identical servers, for example, can be configured with fail-safes so that one will take over if the other fails or requires maintenance. A good rule of thumb is to have redundant components and services in place for any part of a network that cannot be down for more than an hour. For example, if an organization hosts its Web servers or cannot be without Internet connectivity, a second connection should be in place. Having an extra switch, wireless router, and a spare laptop onsite is a good practice for ensuring that downtime is minimal.
3. **Standardization of Hardware and Software:** Standardizing the hardware and software used in a network is essential for ensuring the network runs smoothly. It also reduces costs associated with maintenance, updates, and repairs. Conducting a full audit of the current computer systems, software, and peripherals will help to determine which should be standardized. For example, a CEO or director may require special consideration, but if 90 percent of the employees use the same notebooks, with the same word processing and email programs, a software or hardware patch across the organization can be conducted much less expensively than if everyone used a different computer model with different software installed on each.

4. **Disaster Recovery Plan:** A detailed disaster recovery plan should be a part of any network design. This includes but is not limited to, provisions for backup power and what procedures should be followed if the network or server crashes. It should also include when data is backed up, how it is backed up, and where copies are stored. A comprehensive disaster recovery plan includes office, building, and metropolitan-wide disasters. In most cases, important data should be backed up daily. Many organizations do a full weekly backup, with daily incremental backups that copy any files modified since the last weekly backup. Backup files should be stored in a secure location offsite in the event of a building disaster, such as a fire.
5. **Future Growth of the Organization:** While it is not always possible to anticipate how large an organization maybe five years later, some allowances for future growth must be built into the network design. For example, Microsoft's Small Business Server can be an excellent choice for many small organizations. However, if your office already has sixty employees, Small Business Server could soon be a wasted investment, as it has a limit of only 75 users. The network design should factor in at least 20 percent annual growth, including everything from switch ports to data backup systems.

7 Computer network

7.1 What is a computer network?

A computer network is a group of interconnected nodes or computing devices that exchange data and resources with each other. A network connection between these devices can be established using cable or wireless media. Once a connection is established, communication protocols such as TCP/IP, Simple Mail Transfer Protocol, and Hypertext Transfer Protocol are used to exchange data between the networked devices. A computer network can be as small as two laptops connected through an Ethernet cable or as complex as the Internet, a global computer network system. A computer network must be physically and logically designed to make it possible for the underlying network elements to communicate with each other. This layout of a computer network is known as the computer network architecture.

7.2 Core components of a computer network

Computer network components comprise physical parts and the software required for installing computer networks, both at organizations and at home. The hardware components are the server, client, peer, transmission medium, and connecting devices. The software components are the operating System and protocols.

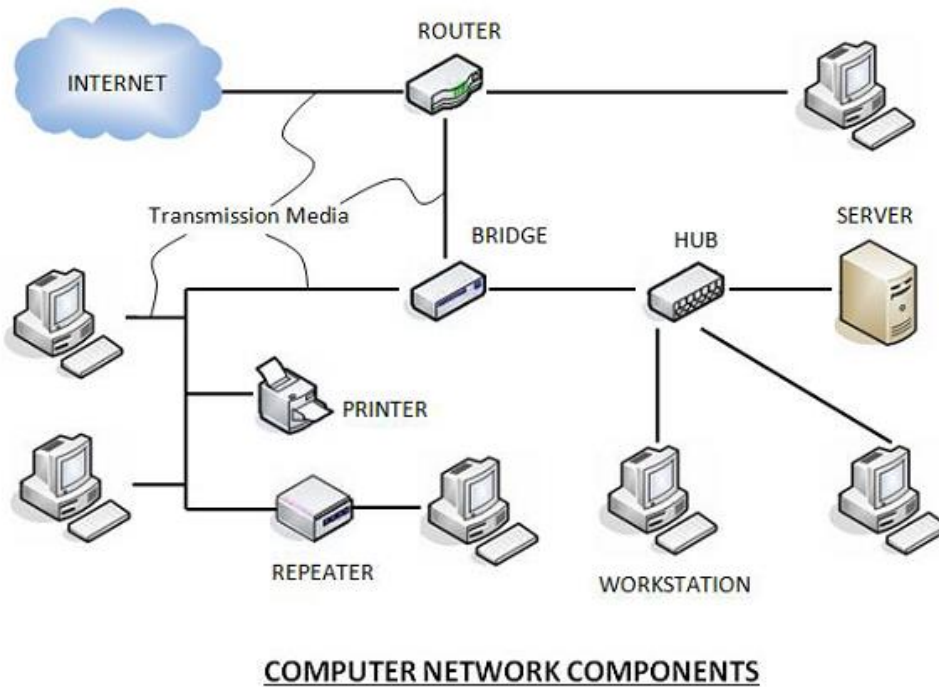


Figure 1: Network along with its components

7.3 Hardware Components

- **Servers:** Servers are high-configuration computers that manage the resources of the network. The network operating system is typically installed in the server, giving users access to the network resources. Servers can be of Figure 1. various kinds: file, database, print, etc.
- **Clients:** are computers that request and receive service from the servers to access and use the network resources.
- **Peers:** Peers are computers that provide and receive services from other peers in a workgroup network.
- **Transmission Media:** Transmission media are the channels through which data is transferred from one device to another in a network. Transmission media may be guided media like coaxial cable, fiber optic cables, or unguided media like microwaves, infrared waves, etc.
- **Connecting Devices:** Connecting devices act as middleware between networks or computers by binding the network media. Some of the common connecting devices are:
 - a. Routers
 - b. Bridges

- c. Hubs
- d. Repeaters
- e. Gateways
- f. Switches

7.4 Software Components

- **Networking Operating System** – Network Operating Systems is typically installed in the server and facilitate workstations in a network to share files, database, applications, printers, etc.
- **Protocol Suite** – A protocol is a computer's rule or guideline for data communication. A protocol suite is a set of related protocols that are laid down for computer networks. The two popular protocol suites are –
 - a. OSI Model (Open System Interconnections)
 - b. TCP / IP Model

8 Advantages of using a computer network

Computer networks are ideal for the quick exchange of information and the efficient use of resources.

The following are the benefits of using a computer network:

1. **Resource sharing.** Enterprises of all sizes can use a computer network to share resources and critical assets. Resources for sharing can include printers, files, scanners and photocopy machines. Computer networks are especially beneficial for larger and globally spread-out organizations, as they can use a single common network to connect with their employees.
2. **Higher connectivity.** Thanks to computer networks, people can stay connected regardless of their location. For example, video calling and document-sharing apps, such as Zoom and Google Docs, enable employees to connect and collaborate remotely.
3. **Data security and management.** In a computer network, data is centralized on shared servers. This helps network administrators to better manage and protect their company's critical data assets. They can perform regular data backups and enforce security measures, such as multifactor authentication, across all devices collectively.

4. **Storage capacity.** Most organizations scale over time and have an abundance of data that needs storage. Computer networks, especially those that employ cloud-based technologies, can store massive amounts of data and backups on a centralized remote server that's accessible to everyone at any given time.
5. **Entertainment.** Computer networks, especially the Internet, offer various sources of entertainment, ranging from computer games to streaming music and videos. Multiplayer games, for example, can only be operated through a local or home-based LAN or a wide area network (WAN), such as the Internet.

Chapter 2: Linux Operating System

Back in the 1950s, computers used to be as enormous as houses? So you can already imagine how cumbersome it must have been to operate them. Furthermore, each computer had its operating System that was fundamentally different from many other computers, making it even more challenging to operate on different machines at once.

In 1969, a team of engineers at Bell Labs decided to work on a standardized operating system known as "Unix." It stood out from other operating systems due to its simple codebase and use of the 'C' programming language rather than assembly language.

In 1991, Linus Torvalds (a student at Helsinki University) decided to work on a freely accessible version of Unix known as Linux; up until that moment, Unix had been reserved exclusively for government entities and major financial enterprises, leaving the populace with essentially the same problems that had led to Unix's rise in the first place. So Torvalds defined a Kernel as a component of Linux and released on October 5, 1991, by Linus Torvalds.

Linux is a Unix-like computer operating system assembled under free, open-source software development and distribution models. Linux was initially developed as a free operating system for Intel x86-based personal computers. It has since been ported to more computer hardware platforms than any other operating system. It is a leading operating system on servers and other big iron systems such as mainframe computers and supercomputers. More than 90% of today's 500 fastest supercomputers run some variant of Linux, including the 10 fastest. Linux also runs on embedded systems such as mobile phones, tablet computers, network routers, televisions, and video game consoles; the Android system widely used on mobile devices is built on the Linux kernel.

1 Basic Features

The following are some of the essential features of the Linux Operating System.

Portable: Portability means software can work on different types of hardware similarly. The Linux kernel and application programs support their installation on any hardware platform.

Open Source: Linux source code is freely available and is a community-based development project. Multiple Teams collaborate to enhance the Linux operating system's capability, which is continuously evolving.

Multiuser: Linux is a multi-user system, meaning multiple users can access system resources like memory/ ram/ application programs simultaneously.

Multiprogramming: Linux is a multiprogramming system that means multiple applications can run simultaneously.

Hierarchical File System: Linux provides a standard file structure for arranging the System Files/ user files.

Shell: Linux provides a unique interpreter program that can be used to execute operating system commands. It can be used for various operations, call application programs, etc.

Security: Linux provides user security using authentication features like password protection/controlled access to specific files/data encryption.

2 Linux Advantages

Low cost: You don't need to spend time and money to obtain license since Linux and much of its software come with the GNU General Public License. You can start to work immediately without worrying that your software may stop working any time because the free trial version expires. Additionally, there are large repositories from which you can freely download high-quality software for almost any task.

Stability: Linux doesn't need to be rebooted periodically to maintain performance levels. It doesn't freeze or slow down over time due to memory leaks. Continuous up-times of hundreds of days (up to a year or more) are not uncommon.

Performance: Linux provides persistent high performance on workstations and networks. It can handle huge numbers of users simultaneously and make old computers sufficiently responsive to be useful again.

Network friendliness: Linux was developed by a group of programmers over the Internet and has strong support for network functionality; client and server systems can be easily setup on any computer running Linux. It can perform tasks such as network backups faster and more reliably than alternative systems.

Flexibility: Linux can be used for high-performance server applications, desktop applications, and embedded systems. You can save disk space by only installing the components needed for a particular use. In addition, you can restrict the use of specific computers by installing only selected office applications instead of the whole suite.

Compatibility: It runs all common UNIX software packages and can process all common file formats.

Choice: The large number of Linux distributions gives you an alternative. Each distribution is developed and supported by a different organization. You can pick the one you like best; the core functionalities are the same, and most software runs on most distributions.

Fast and easy installation: Most Linux distributions have user-friendly installation and setup programs. Popular Linux distributions come with tools that make installing additional software very user-friendly.

Full use of hard disk: Linux works well even when the hard disk is almost full.

Multi-tasking: Linux is designed to do many things simultaneously; e.g., a large printing job in the background won't slow down your other work.

Security: Linux is one of the most secure operating systems. Walls and flexible file access permission systems prevent unwanted visitors or virus access. Linux users can select and safely download software free of charge from online repositories containing thousands of high-quality packages. No purchase transactions requiring credit card numbers or other sensitive personal information are necessary.

Open Source: If you develop software that requires knowledge or modification of the operating system code, LINUX's source code is at your finger tips. Most Linux applications are Open Source as well.

3 Drawbacks of Linux

Hardware drivers: Most of the users of Linux face an issue while using Linux. Various hardware companies prefer to build drivers for Mac or Windows because they contain several users than Linux. Linux has small drivers for peripheral hardware than Windows.

Software alternative: Let's take Photoshop example, a famous graphic editing tool. Photoshop exists for Windows; however, it is not available on Linux. Also, there are some other tools for photo editing, but the Photoshop tool is more powerful as compared to others. Another example is MS Office which is not present for Linux users.

Learning curve: Linux isn't an exceptionally user-friendly operating system. Hence, it might not be very clear for many beginners. On the other hand, beginning Windows is efficient and easy for many beginners; however, understanding Linux works is complex. We must understand the command line interface, and finding newer software is tricky. When we face any issue in the OS, the search solution is very problematic. Also, there are various experts for Mac and Windows as compared to Linux.

Games: Several games are developed for Windows but, unfortunately, not for Linux. Because the Windows platform is used widely, so, the developers of the games are more interested in Windows.

4 Linux System Architecture

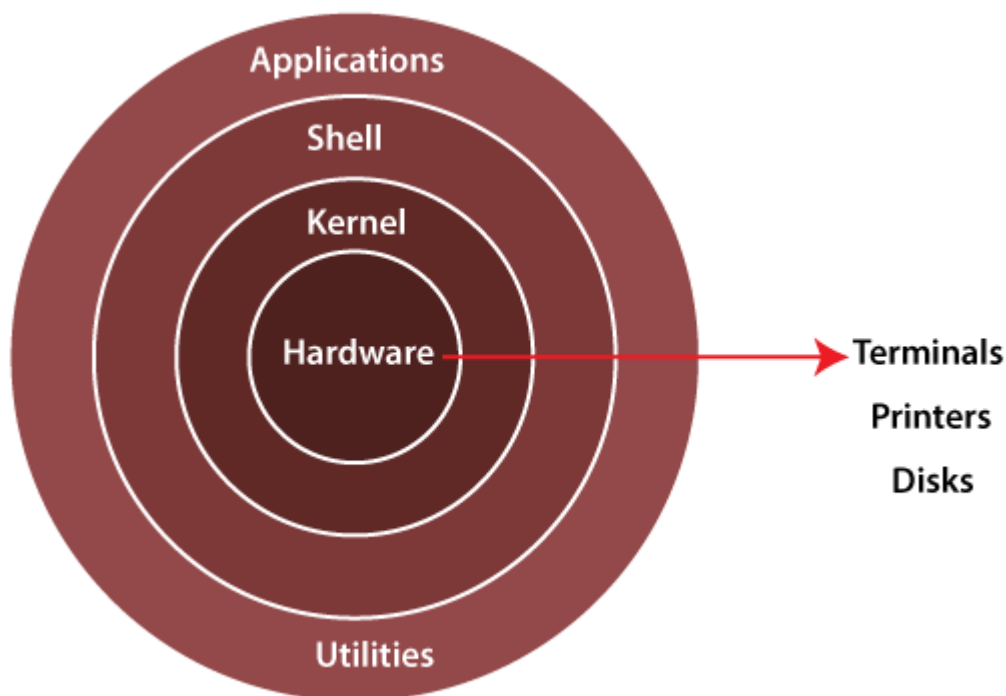


Figure 2. Linux operating system architecture

- A. **Hardware:** physical parts of a computer, such as central processing unit (CPU), monitor, mouse, keyboard, hard disk, and other connected devices to the CPU
- B. **Kernel:** Kernel is a small and special code that is the core component of Linux OS and directly interacts with hardware. The intermediate level between software and hardware provides low-level service to the user mode's components. It is fully developed in C language and file system architecture. Moreover, it has different blocks which manage various operations. In this tutorial, we will learn about the kernel architecture of Linux. The Kernel runs several processes concurrently and manages the various resources. It is viewed as a resource manager when several programs run concurrently on a system. In this case, the Kernel is an instance that shares available resources like CPU time, disk space, network connections, etc.

A kernel is a computer program and an operating system's central core. It manages the operations of the computer and the hardware, most notably memory and CPU time. It is an integral part of any operating system.

1. The Kernel is the core (main) of the Linux operating system.
 2. It is a collection of routine communication with hardware directly.
 3. It loads into memory when Linux is booted.
 4. The Kernel provides support to user programs through a system call.
 5. The Kernel manages Computer memory, schedules process, decides processes' priorities, and performs other tasks.
 6. The Kernel does a lot of work even if no application software runs. Hence Kernel is often called an application software gateway to computer resources.
 7. The Kernel is represented by */boot*.
- C. **Shell:** is an environment where we can run our commands, programs, and shell scripts. It is a user interface for access to an operating system's services. (User interface program execution, file system manipulation, input/output operations, communication, resource allocation, error detection, security, and protection)

1. Shell is an interface between the user and Kernel.
2. It is the outer part of the operating System.
3. A shell is a user interface to access an operating system's services. Shell is an environment where we can run commands, programs, software, and shell scripts.
4. Computers do not have any inherent capability of translating commands into actions; Shell does it.
5. There can be many shells in action - one for each logged-in user.

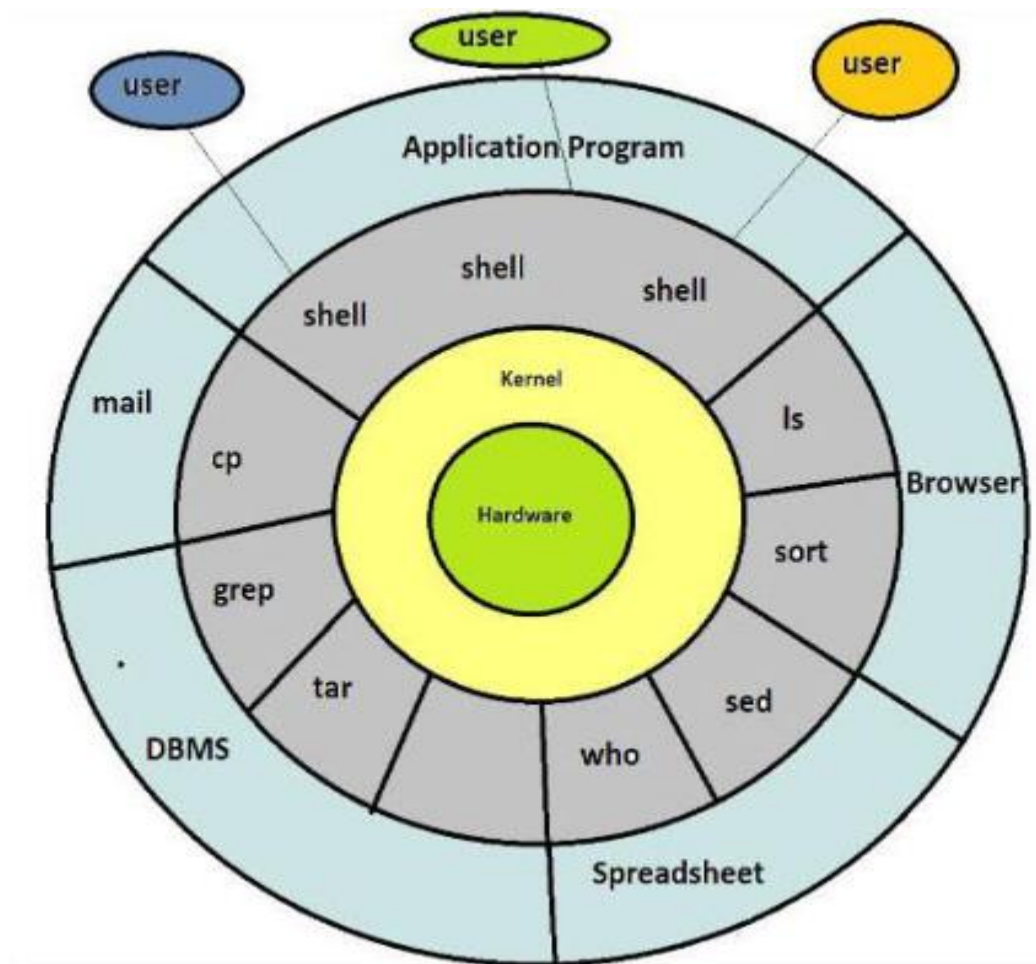


Figure3. The diagram of kernel shell user relationship

How does Shell work?

When we enter commands through the keyboard, it gathers input from us and executes programs based on that input. When a program finish is executing, it displays that program's output. OR the shell thoroughly examines the keyboard input for special characters. If it finds any, it rebuilds a simplified command line and finally communicates with the Kernel to see that the command is executed. Use the echo \$SHELL command in the terminal to know the running shell.

- D. **Application programs/software:** An application, or application program, is a software program that runs on your computer. It is exciting for the user. Some inbuilt application programs in Linux are terminal, Firefox browser, and Libre Office System calls. There are over a thousand commands available in the Linux operating system. All these commands use a function to communicate with Kernel - and it is called a system call. The system call is the interface between a process and an operating system, or System calls are the only entry points into the kernel system.

E.g.

- (1) A typical Linux writes a file with the write system call. The same system call can access both a file and a device.
 - (2) Open system call opens both file and device. These system calls are built into Kernel. And interaction through the system calls represents efficient communication with the operating System.
- E. **System utility:** This layer consists of programs that perform various system-level tasks, such as managing processes, controlling user accounts, and configuring system settings. These utilities are usually command-line programs invoked by the user or other programs.

5 Linux directory structure

If you are faintly acquainted with Linux, you might have heard the terms root, lib, bin, etc. These are various directories that you'll find in all Linux distributions. The Linux Foundation maintains a Filesystem Hierarchy Standard (FHS). This FHS defines the directory structure and the

content/purpose of the directories in Linux distributions. Thanks to this FHS, you'll find the same directory structure in (almost) all the Linux distributions.

Linux is based on UNIX, so it borrows its filesystem hierarchy from UNIX. So you'll find a similar directory structure in UNIX-like operating systems such as BSD and macOS. I'll be using the term Linux hereafter instead of UNIX, though.

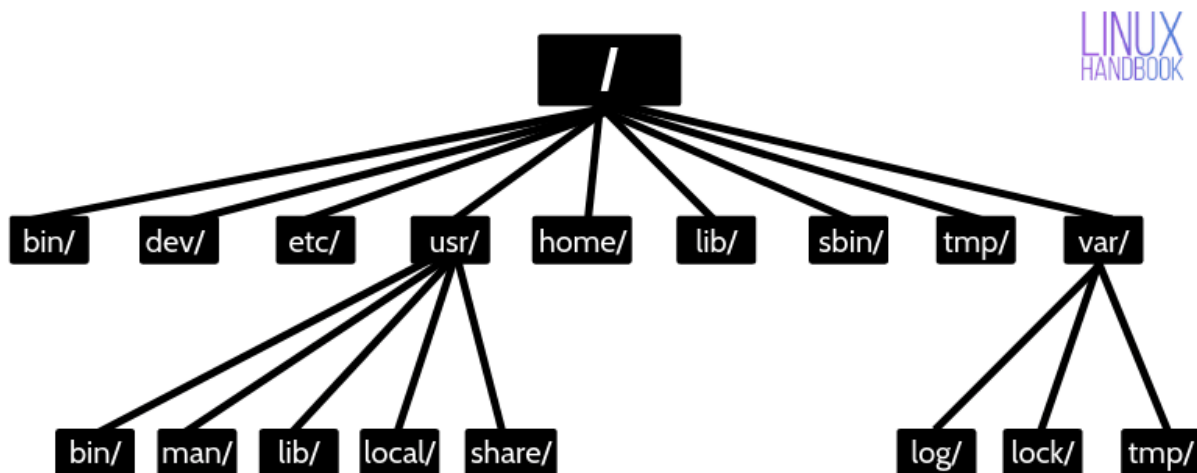


Figure 4. Linux Directory Structure

Since all other directories or files are descended from the root, the absolute path of any file is traversed through the root. For example, if you have a file in /home/user/documents, you can guess that the directory structure goes from root->home->user->documents.

/bin - Binaries

The '/bin' directly contains the executable files of many basic shell commands like `ls`, `cp`, `cd`, etc. Mostly the programs are in binary format here and accessible by all the users in the Linux system.

/dev – Device files

The `/dev/` directory consists of files representing devices attached to the local System. However, these are not regular files that a user can read and write to; these files are called devices files or particular files:

/etc – Configuration files

The `/etc` directory contains the System's core configuration files, primarily used by the administrator and services, such as the password file and networking files. If you need to make changes in system configuration (for example, changing the hostname), this is where you'll find the respective files.

/usr – User binaries and program data

in `/usr` go all the executable files, libraries, and sources of most system programs. For this reason, most of the files contained therein are read-only (for the normal user)

- `/usr/bin` contains executable files for basic user commands
- `/usr/sbin` contains additional commands for the administrator
- `/usr/lib` contains the system libraries files for user programs
- `/usr/share` contains documentation or is common in libraries. For example `/usr/share/man` contains the text of the manpage

/home – User personal data.

The home directory contains personal directories for the users. The home directory contains the user data and user-specific configuration files. You'll put your personal files, notes, programs, etc in your home directory as a user.

When you create a user on your Linux system, it's a general practice to create a home directory for the user. For example, suppose your Linux system has two users, Alice and Bob. They'll have a home directory at locations `/home/alice` and `/home/bob`.

Do note that Bob won't have access to /home/alice and vice versa. That makes sense because only the user should access their home.

/lib – Shared libraries

Libraries are codes that the executable binaries can use. For example, the /lib directory holds the libraries needed by the binaries in /bin and /sbin directories.

Libraries needed by the binaries in the /usr/bin and /usr/sbin are located in the directory /usr/lib.

/sbin – System binaries

This is similar to the /bin directory. The only difference is that it contains the binaries that can only be run by root or a sudo user. You can think of the 's' in 'sbin' as super or sudo.

/tmp – Temporary files

As the name suggests, this directory holds temporary files. Many applications use this directory to store temporary files. Even you can use a directory to store temporary files.

But do note that the contents of the /tmp directories are deleted when your system restarts. Some Linux systems also delete old files automatically, so don't store anything important here.

/var – Variable data files

Var, short for variable, is where programs store runtime information like system logging, user tracking, caches, and other files that system programs create and manage.

The files stored here are NOT cleaned automatically, providing a good place for system administrators to look for information about their system behavior. For example, if you want to check the login history in your Linux system, check the file's content in /var/log/wtmp.

/root – The home directory of the root

There is /root directory, which also works as the root user's home directory. So instead of /home/root, the root's home is located at /root. Please do not confuse it with the root directory (/).

/media – Mount point for removable media

When you connect a removable media such as a USB disk, SD card, or DVD, a directory is automatically created under the /media directory for them. You can access the content of the removable media from this directory.

/mnt– Mount directory

This is similar to the /media directory, but instead of automatically mounting the removable media, mnt is used by system administrators to mount a filesystem manually.

/srv – Service data

The /srv directory contains data for services provided by the System. For example, if you run an HTTP s/mntserver, storing the website data in the /srv directory is a good practice.

I think this information is enough for you to understand the Linux directory structure and its usage. In the end, if you want, you can download and save this image for quick reference to the directory structure in Linux systems.

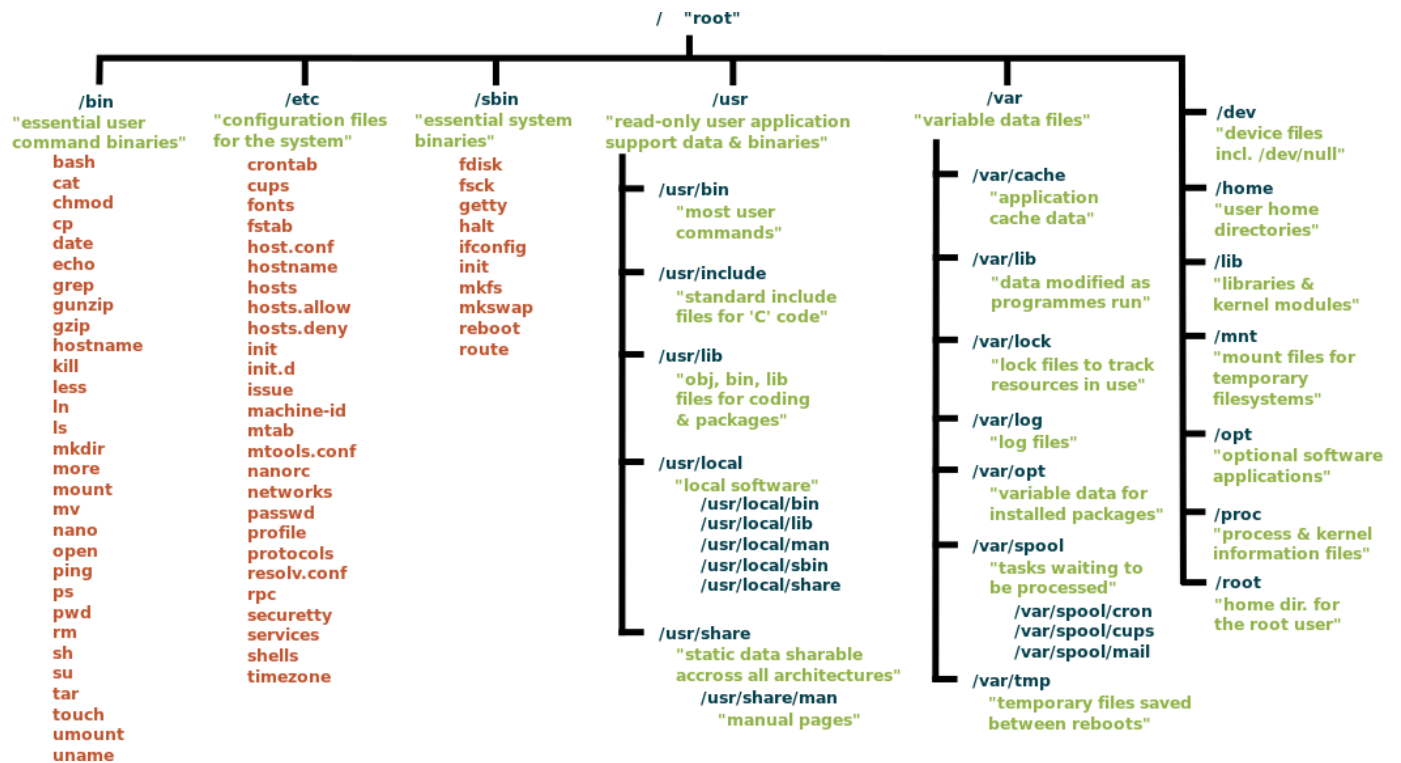


Figure 5. Linux directory structure and descriptions summary

6 Linux directories and File color

- **Blue:** Directory
- **Green:** Executable or recognized data file
- **Cyan (Sky Blue):** Symbolic link file
- **Yellow with black background:** Device
- **Magenta (Pink):** Graphic image file
- **Red:** Archive file
- **Red with black background:** Broken link
- **White Files**

```
*
├── directory_name
├── helloworld.c
├── helloworld_exe
├── helloworld_softlink -> helloworld_exe
├── helloworld_softlink_broken -> helloworld_exe1
├── hello.zip
├── mypicture.jpg
└── textfile.txt

1 directory, 7 files
```

Figure 6. Linux directories and file color

```
stefano@3000-G530:~$ python test.py
directory
symbolic link.
pipe
socket
door
block device driver
character device driver
symlink to nonexistent file, or non-stat'able file
file that is setuid (u+s)
file that is setgid (g+s)
file with capability
dir that is sticky and other-writable (+t,o+w)
dir that is other-writable (o+w) and not sticky
dir with the sticky bit set (+t) and not other-writable
executable files
```

Figure 7. Linux directories and file color and background color

7 Ubuntu Linux Commands

Commands are generally followed by one or more options, followed by arguments. Options describe the command behavior, and arguments are usually files and folders. Every command is associated with its options. The general format for commands looks like this

\$ Command <options><arguments>

Linux commands options and arguments examples

Let us consider the *ls*(List directory content) command. The **ls command shows files and folders of the current working directory**. The most commonly used options that come with the *ls* command are

Option	Description
-a or --all	Lists all files and directories, including hidden ones.
-l	Display results in a detailed/long format.
-h or --human-readable	Display file sizes in human-readable format rather than in bytes.
-S (Capital Letter)	Sort results by file size.
-t	Sort results by modification date.
-r or --reverse	Sort results in reverse order.

Examples: Let's navigate to the home directory and issue the *ls* command

1. **ls** without Options

The command displays the files and folders, excluding hidden ones

```
kijazi@Ubuntu:~$ ls
Desktop  Downloads  Music      Public     Videos
Documents examples.desktop Pictures    Templates
```

2. **ls** with **-a** or **-all** options

Display the files and folders, including hidden ones.

```
kijazi@Ubuntu:~$ ls -a
.      Desktop      .ICEauthority .sudo_as_admin_successful
..     .dmrc        .local        Templates
.bash_history Documents    .mozilla      Videos
.bash_logout Downloads    Music         .Xauthority
.bashrc   examples.desktop Pictures       .xsession-errors
.cache    .gconf      .profile     .xsession-errors.old
.config   .gnupg     Public
```

3. **ls** with **-R**

The **"-R"** flag will print subdirectories of a directory

```
kijazi@Ubuntu:~$ ls -R
.:
Desktop  Downloads  medy  Pictures  Templates
Documents examples.desktop Music  Public  Videos

./Desktop:
BIT

./Desktop/BIT:

./Documents:

./Downloads:

./medy:
juma

./medy/juma:

./Music:

./Pictures:
Screenshot from 2023-05-08 07-04-46.png

./Public:

./Templates:

./Videos:
kijazi@Ubuntu:~$
```

4. ls with -all

Print the detailed information of the files/directories (including hidden), such as directory owner, permission, and the date created.

```
kijazi@Ubuntu:~$ ls -all
total 116
drwxr-xr-x 17 kijazi kijazi 4096 Cam 9 07:04 .
drwxr-xr-x  4 root   root   4096 Cam 4 13:22 ..
-rw-r--r--  1 kijazi kijazi 1723 Cam 8 19:24 .bash_history
-rw-r--r--  1 kijazi kijazi  220 Cam 4 13:22 .bash_logout
-rw-r--r--  1 kijazi kijazi 3771 Cam 4 13:22 .bashrc
drwx----- 14 kijazi kijazi 4096 Cam 9 07:06 .cache
drwx----- 15 kijazi kijazi 4096 Cam 4 15:38 .config
drwxr-xr-x  3 kijazi kijazi 4096 Cam 8 19:15 Desktop
-rw-r--r--  1 kijazi kijazi   25 Cam 4 15:31 .dmrc
drwxr-xr-x  2 kijazi kijazi 4096 Cam 4 15:32 Documents
drwxr-xr-x  2 kijazi kijazi 4096 Cam 4 15:32 Downloads
-rw-r--r--  1 kijazi kijazi 8980 Cam 4 13:22 examples.desktop
drwx-----  2 kijazi kijazi 4096 Cam 6 17:44 .gconf
drwx-----  3 kijazi kijazi 4096 Cam 9 07:04 .gnupg
-rw-r--r--  1 kijazi kijazi 1272 Cam 9 07:04 .ICEauthority
drwx-----  3 kijazi kijazi 4096 Cam 4 15:32 .local
drwxrwxr-x  3 kijazi kijazi 4096 Cam 8 07:27 medy
drwx-----  4 kijazi kijazi 4096 Cam 4 15:37 .mozilla
drwxr-xr-x  2 kijazi kijazi 4096 Cam 4 15:32 Music
drwxr-xr-x  2 kijazi kijazi 4096 Cam 8 07:05 Pictures
-rw-r--r--  1 kijazi kijazi  655 Cam 4 13:22 .profile
```

More Linux Commands

pwd (present working directory)

This command refers to the present working directory in which you are operating. It shows the path from the root to the current working directory.

```
kijazi@Ubuntu:~$ pwd
/home/kijazi
kijazi@Ubuntu:~$
```

dir

The dir command is used to print (on the terminal) all the available directories in the present working directory:

```
kijazi@Ubuntu:~$ dir
Desktop    Downloads      medy  Pictures  Templates
Documents  examples.desktop Music  Public    Videos
kijazi@Ubuntu:~$
```

cd (Change directory Command)

One of the most used commands of Ubuntu; you can change the directories in the terminal using the "cd" command. For instance, the following command will change the present working directory to Desktop.

```
kijazi@Ubuntu:~$ ls
Desktop  Downloads      medy  Pictures  Templates
Documents examples.desktop Music  Public   Videos
kijazi@Ubuntu:~$ cd Desktop
kijazi@Ubuntu:~/Desktop$
```

cd /

You can navigate to any directory from the present working directory by specifying the path by imitating the root directory using the cd command. For example, the user navigates from the Desktop to the var directory in the following command.

```
kijazi@Ubuntu:~/Desktop$ cd /var
kijazi@Ubuntu:/var$
```

How to Create, Read, Edit, and Delete Files in Ubuntu

touch

This Ubuntu command can be used to create a new file as well one can use it to change the timestamp of any file; the command given below will create a new text file **programming.txt** in pwd which is a desktop:

```
kijazi@Ubuntu:~/Desktop$ touch programming.txt
kijazi@Ubuntu:~/Desktop$ ls
BIT  programming.txt
kijazi@Ubuntu:~/Desktop$
```

If we execute a touch command to create a file, but the file is already created, then it would change the timestamp of that file to the current time; for instance, the command given below will change the timestamp of the **programming.txt**. You can check that the timestamp has been changed to the current time:

```
kijazi@Ubuntu:~/Desktop$ touch programming.txt
kijazi@Ubuntu:~/Desktop$ stat programming.txt
  File: 'programming.txt'
  Size: 0          Blocks: 0          IO Block: 4096   regular empty file
Device: 806h/2054d Inode: 522768       Links: 1
Access: (0664/-rw-rw-r--)  Uid: ( 1000/  kijazi)   Gid: ( 1000/  kijazi)
Access: 2023-05-10 19:08:08.514751999 +0300
Modify: 2023-05-10 19:08:08.514751999 +0300
Change: 2023-05-10 19:08:08.514751999 +0300
 Birth: -
kijazi@Ubuntu:~/Desktop$
```

nano

Nano is a text editor for editing documents. Now let us open the file using the nano text editor and Type Network and System Administration. We can Close the file with a shortcut key **CTRL+X**; the terminal will allow you to save or ignore the file.

```
kijazi@Ubuntu:~/Desktop$ nano programming.txt
```

```
GNU nano 2.5.3      File: programming.txt      Modified
Network and System Administration

```

^G Get Help	^O Write Out	^W Where Is	^K Cut Text	^J Justify	^C Cur Pos
^X Exit	^R Read File	^_ Replace	^U Uncut Text	^T To Spell	^_ Go To Line

cat

This command is used to show the content of any file: For instance, the following command will display the content inside "**programming.txt**".

```
kijazi@Ubuntu:~/Desktop$ cat programming.txt
System and Network Administrator
```

Also, it is possible to copy the contents of one file or more files to another using the **cat** command.

```
kijazi@Ubuntu:~/Desktop$ cat programming.txt programming2.txt>output.txt
```

mkdir

The above command will make a directory in your **pwd**(present working directory); for example, the following command will make the directory "**BIT**" in **pwd**.

```
kijazi@Ubuntu:~/Desktop$ mkdir BIT
kijazi@Ubuntu:~/Desktop$ ls
BIT  output.txt  programming.txt
kijazi@Ubuntu:~/Desktop$
```

rm

This remove command is used to remove the specific file from a directory; For instance, below mentioned command would remove the "**output.txt**" file from the **pwd**:

```
kijazi@Ubuntu:~/Desktop$ ls
BIT  output.txt  programming.txt
kijazi@Ubuntu:~/Desktop$ rm output.txt
kijazi@Ubuntu:~/Desktop$ ls
BIT  programming.txt
kijazi@Ubuntu:~/Desktop$
```

rmdir

Or you can remove the empty directory, as the command given below will remove the "**BIT**" directory:

```
kijazi@Ubuntu:~/Desktop$ ls
BIT  programming.txt
kijazi@Ubuntu:~/Desktop$ rmdir BIT
kijazi@Ubuntu:~/Desktop$ ls
programming.txt
kijazi@Ubuntu:~/Desktop$
```

rm -r

If the directory still contains files or subdirectories, the `rmdir` command does not remove the directory.

To remove a directory and all its contents, including any subdirectories and files, use the `rm` command with the recursive option, **-r**.

```
kijazi@Ubuntu:~/Desktop$ ls
examples.desktop  medy  Pictures  programming.txt
kijazi@Ubuntu:~/Desktop$ rm -r medy
kijazi@Ubuntu:~/Desktop$ ls
examples.desktop  Pictures  programming.txt
kijazi@Ubuntu:~/Desktop$
```

Note that: (1). You can also remove multiple directories using

```
$ rm -r directory1 directory2 ....directoryn
```

If you do not have write permissions on the directory and contents you wish to delete, you will need to use root privileges or log in to the correct user account that has permissions on the directory. For example, you can use `sudo` like so:

```
$ sudo rm -r medy
```

cp

The `cp` command will help you to copy any file or folder to any directory. Example the command below copy the file "**examples.desktop**" to the directory "**Desktop**"

```
kijazi@Ubuntu:~$ ls
Desktop  Downloads  medy  Pictures  Templates
Documents  examples.desktop  Music  Public  Videos
kijazi@Ubuntu:~$ cp examples.desktop Desktop
kijazi@Ubuntu:~$ ls
Desktop  Downloads  medy  Pictures  Templates
Documents  examples.desktop  Music  Public  Videos
kijazi@Ubuntu:~$ cd Desktop
kijazi@Ubuntu:~/Desktop$ ls
examples.desktop  programming.txt
kijazi@Ubuntu:~/Desktop$
```

Using `cpit` is possible to copy one directory and its content to another directory. The following example copy the folder **Pictures** to folder **Desktop**

```
kijazi@Ubuntu:~$ ls
Desktop  Downloads  medy  Pictures  Templates
Documents  examples.desktop  Music  Public  Videos
kijazi@Ubuntu:~$ cp Pictures -r Desktop
kijazi@Ubuntu:~$ ls Desktop
examples.desktop  Pictures  programming.txt
kijazi@Ubuntu:~$ cd Desktop
```

mv

You can use this command to move files/folders around the computer, and you can also rename files or directories inside a specific directory: the command given below will move the directory "**medy**" to "**Pictures**":

```
kijazi@Ubuntu:~$ ls
Desktop  Downloads      medy  Pictures  Templates
Documents examples.desktop Music  Public    Videos
kijazi@Ubuntu:~$ mv medy Desktop
kijazi@Ubuntu:~$ ls
Desktop  Downloads      Music  Public    Videos
Documents examples.desktop Pictures  Templates
kijazi@Ubuntu:~$ ls Desktop
examples.desktop  medy  Pictures  programming.txt
kijazi@Ubuntu:~$
```

wget

You can use the **wget** command to download the content from the internet; for instance, the following command will download VirtualBox.

```
adnan@adnan:~$ wget https://download.virtualbox.org/virtualbox/6.1.26/VirtualBox-6.1.26-145957-Win.exe
--2021-08-12 18:43:30-- https://download.virtualbox.org/virtualbox/6.1.26/VirtualBox-6.1.26-145957-Win.exe
Resolving download.virtualbox.org (download.virtualbox.org)... 92.123.108.54
Connecting to download.virtualbox.org (download.virtualbox.org)|92.123.108.54|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 108235504 (103M) [application/octet-stream]
Saving to: 'VirtualBox-6.1.26-145957-Win.exe'

VirtualBox-6.1.26-14  3%[          ]  3.79M  1.39MB/s
```

history

The history command shows the list of commands (with numeric numbers) executed:

```
kijazi@Ubuntu:~/Desktop$ cd ..
kijazi@Ubuntu:~$ history
 1 cd etc
 2 cd /etc
 3 ls
 4 cd cd /boot
 5 cd /boot
 6 ls
 7 cd vmlinuz
 8 cd ..
 9 SHELL
10 sudo shell
11 sudo ls
12 sudo root
13 su root
14 pwd
15 su root
16 sudo cd /var
17 su root
```

grep

With the help of grep, you can search for a pattern in which a specific word lies; for instance, the command given below will print all the lines that contain "20" from "file1.txt"

```
adnan@adnan:~$ cat file1.txt | grep 20  
Ford Escape SEL- 29205  
adnan@adnan:~$
```

:

8 Ubuntu Software Repositories

If you've used Ubuntu, you might have noticed various software repositories. Here's an explanation of them. While installing software on Ubuntu using the command line, you might have seen the word "repository" often used in the output. If you're new to the whole Linux universe, this might be a new term for you. What does it mean, and why does your System need these repositories? Therefore, this part introduces the concept of repositories in Ubuntu, along with a brief description of the available repositories.

Generally speaking, a repository is a collection of multiple items stored together in enormous quantities. On Ubuntu and other Unix-based operating systems, a repository refers to a ginormous collection of software and packages you can install on your System.

Unlike Windows and macOS, Linux provides software to its users in a well-packaged format, which differs across different distributions. For example, Debian-based distributions rely on DEB packages. Similarly, you will find RPM packages on Fedora, CentOS, and other RHEL-based distros.

Also, different Linux distros have their own set of repositories. For example, the default ones will belong to Ubuntu itself on Ubuntu. Apart from these, users can also add any of their choices by using the **add-apt-repository** command.

The recommended way to install packages on Ubuntu is using the official repositories. This is because the packages in these repositories are specially developed for Ubuntu. Also, regular updates pushed by the developers ensure that the software works properly.

8.1 Types of Repositories in Ubuntu

Ubuntu ships with four different types of repositories. Namely, these are Main, Restricted, Universe, and Multiverse. Some, like Main, are open by default. But for others, you must enable Universe and Multiverse before you can start fetching packages from them.

Main

The main includes software and packages that are fully supported by the Ubuntu team. If you've installed software from the main repository, Ubuntu will regularly provide security updates and bug fixes for those packages.

This repository consists of open-source packages that are free to use and redistribute. Also, you'll find that Ubuntu comes with most of the packages in the main repository as they are essential utilities required by the System and the user.

Restricted

Although you can use the software available in Restricted repositories without any charge under a free license, you can't redistribute these packages. The restricted repository includes tools and drivers necessary for the operating System's proper working.

The Ubuntu team doesn't support such programs as they belong to another author. Also, Canonical, the company responsible for managing Ubuntu, can't modify the package as most of the software in the Restricted repository is proprietary.

Universe

As the name suggests, universe contains every open-source package developed for the Linux operating system. However, the Ubuntu team doesn't directly manage these packages. Instead, the community of developers working on a package is solely responsible for pushing updates and security fixes.

However, Ubuntu can move the package from Universe to Main if the developers agree to follow the specific standards they set.

Multiverse

While the repositories mentioned above contain either free-to-use or open-source packages, multiverse includes software that isn't available for free. Proprietary programs with no license or legal issues are also included in the multiverse. Installing packages from this repository is not recommended because the risk associated with these programs is significant.

8.2 Working With Repositories and Packages

Linux gives you complete control over which repository to choose while installing packages. You can either go for the trusted Ubuntu repositories if you want to be on the safe side, or you can download Linux software from the universe or multiverse repository. But that's only suggested if you know what you're doing.

Every Linux distribution has a default package manager responsible for installing, updating, and upgrading the System's packages. For example, Ubuntu comes with **APT(Advanced Package Tool) and dpkg (Debian Package) package managers**, and Fedora Linux uses DNF for managing packages. Similarly, you can install and remove software in Arch Linux using Pacman, the default package manager that ships with the OS.

8.3 What's the Difference Between APT and dpkg in Ubuntu?

APT(Advanced Package Tool) and **dpkg(Debian Package)** are command-line package management interfaces you can use in the terminal on Ubuntu and other Debian-based systems. **dpkg** is a low-level tool, and **APT** is a higher-level tool. **APT** is more commonly used than **dpkg** as it can fetch packages from remote locations and deal with complex package relations, such as dependency resolution.

APT Uses **dpkg** to Install Packages When **APT** (or its cousin, **Apt-get**) installs a package; it uses **dpkg** on the backend to accomplish that. That way, **dpkg** acts more as an "under the hood" tool for **APT**'s more user-friendly interface.

With **APT**, you can retrieve a file from a remote repository and install it all in one command. This saves you from manually finding and downloading the package before installation. With

dpkg, you can only install local files you've downloaded. It can't search remote repositories or pull packages from them.

8.4 Package management with APT and dpkg

Package management via apt-get runs hand-in-hand with the `sources.list` file. The `sources.list` file is a key factor in adding or upgrading applications to your Ubuntu installation. This is also used by your system for system updates. The file is basically the roadmap for your system to know where it may download programs for installation or upgrade.

Ubuntu uses apt for package management. Apt stores a list of repositories or software channels in the file

```
/etc/apt/sources.list
```

and in any file with the suffix `.list` under the directory

```
/etc/apt/sources.list.d/
```

Typically, the beginning of the file `/etc/apt/sources.list` looks like this:

```
# sources.list
#deb cdrom:[Ubuntu 13.10 _Saucy Salamander_ - Release i386 (20131016.1)]/
saucy main restricted

# See http://help.ubuntu.com/community/UpgradeNotes for how to upgrade to
# newer versions of the distribution.
deb http://ch.archive.ubuntu.com/ubuntu/ saucy main restricted
deb-src http://ch.archive.ubuntu.com/ubuntu/ saucy main restricted
#...
```

Explanation of the Repository Format

- All the lines beginning with one or two hashes (#) are comments, for information only.
- The lines without hashes are *apt repository lines*. Here's what they say:

- **deb:** These repositories contain binaries or precompiled packages. These repositories are required for most users.
- **deb-src:** These repositories contain the source code of the packages. Useful for developers.
- **http://archive.ubuntu.com/ubuntu:** The URI (Uniform Resource Identifier), indicates the package location in the internet
- **saucy** is the release name (code name) of your ubuntu. For example, Ubuntu **Ubuntu 22.04 LTS is also known as JammyJellyfish** and Ubuntu 20.04 LTS is a *Focal Fossa*

Note: To know your current Ubuntu release name (code name) `lsb_release -sc`

Note: To know your current Ubuntu version and release name run the command `lsb_release -a`

- **main&restricted:** This is repository types

Note: It's always a good idea to backup a configuration file like **sources.list** before you edit it. To do so, issue the following command:

```
sudo cp /etc/apt/sources.list /etc/apt/sources.list.backup
```

8.5 Adding Repositories

Additional software repositories such as Universe and Multiverse can be enabled by uncommenting the corresponding apt lines (i.e., delete the '#' at the beginning of the line). For Universe, uncomment those lines:

```
deb http://us.archive.ubuntu.com/ubuntu/ saucy universe
deb-src http://us.archive.ubuntu.com/ubuntu/ saucy universe
deb http://us.archive.ubuntu.com/ubuntu/ saucy-updates universe
deb-src http://us.archive.ubuntu.com/ubuntu/ saucy-updates universe
```

There are four similar lines for 'multiverse'.

OR you may use the ***add-apt-repository*** command. If your release is 'saucy':

```
sudo add-apt-repository "deb http://us.archive.ubuntu.com/ubuntu/ saucy
universe multiverse"
sudo add-apt-repository "deb http://us.archive.ubuntu.com/ubuntu/ saucy-
updates universe multiverse"
```

Don't forget to retrieve the updated package lists:

```
sudo apt-get update
```

Note: Depending on your location, you should replace 'us.' by another country code, referring to a mirror server in your region. Check **sources.list** to see what is used!

8.5.1 Adding Partner Repositories

You can add the partner repositories by uncommenting the following lines in your /etc/apt/sources.list file:

```
deb http://archive.canonical.com/ubuntu saucy partner
deb-src http://archive.canonical.com/ubuntu saucy partner
```

Be aware that the software contained within this repository is NOT open source.

Then update as before:

```
sudo apt-get update
```

8.5.2 Adding Other Repositories

There are some reasons for which you might want to add non-Ubuntu repositories to your list of software sources. **Caution:** To avoid trouble with your system, only add repositories that are trustworthy and that are known to work on Ubuntu systems!

You can add custom software repositories by adding the apt repository line of your software source to the end of the **sources.list** file. It should look something like this:

```
deb http://mirror3.ubuntulinux.nl/ hardy-seveasfreenx
```

8.5.3 Adding Launchpad PPA Repositories

Canonical (is a **UK-based privately held computer software company** founded and funded by South African entrepreneur Mark Shuttleworth to market commercial support and related services for Ubuntu and related projects.) today announced the general availability of the Launchpad Personal Package Archive (PPA) service, a new way for developers to build and publish packages of their code, documentation, artwork, themes and other additions to the Ubuntu environment on desktop, server and now mobile platforms.

Adding Launchpad PPA (Personal Package Archive) is possible conveniently via the command: **add-apt-repository**. This command is similar to "addrepo" on Debian.

- The command updates your sources.list file or adds/edits files under sources.list.d/. Type **man add-apt-repository** for detailed help.
- If a public key is required and available it is automatically downloaded and registered.
- Should be installed by default. On older or minimal Ubuntu releases, you may have to install **software-properties-common** and/or **python-software-properties** first (sudo apt-get install python-software-properties)

```
sudo add-apt-repository ppa:<repository-name>
```

Example: `sudo add-apt-repository ppa:nhandler/ppa`

8.5.4 Suggestions & Recommendations

- It is always a good idea to **back up** configuration files like `/etc/apt/sources.list` before you begin editing. You can then revert your changes if needed.
- If you decide to add other repositories to `sources.list`, make sure that the repository is **meant to work (and known to work)** with Ubuntu. Repositories that are **not designed to work with your version of Ubuntu** can introduce inconsistencies in your system and might force you to re-install. Also, make sure that you really need to add external repositories as the software package(s) you are looking for may already have been introduced into the official repositories!
- Please keep in mind that it may not be legal (typically because of export restrictions) to enable some *non-Ubuntu* software repositories in some countries.
- You may be asked enter a security key when adding a non-Ubuntu repository to your sources.
- Remember to retrieve updated package lists by issuing a **sudo apt-get update** when you're done editing `sources.list`.

8.5.5 Packages Installation commands

This command installs a new package.

```
apt-get install <package_name>
```

This command searches the repositories and installs the build dependencies for `<package_name>`. If the package is not in the repositories, it will return an error.

```
apt-get build-dep <package_name>
```

APT and aptitude will accept multiple package names as a space delimited list. For example:

```
apt-get install <package1_name><package2_name><package3_name>
```

8.5.6 Maintenance commands

Run this command after changing */etc/apt/sources.list* or */etc/apt/preferences*. Run this command periodically to make sure your source list is up-to-date. This is the equivalent of "Reload" in Synaptic or "Fetch updates" in Adept.

```
apt-get update
```

This command upgrades all installed packages.

```
apt-get upgrade
```

This command is a diagnostic tool. It does an update of the package lists and checks for broken dependencies.

```
apt-get check
```

This command is a diagnostic tool. It does an update of the package lists and checks for broken dependencies.

```
apt-get check
```

This command does the same thing as **Edit->Fix Broken Packages** in Synaptic. Do this if you get complaints about packages with "unmet dependencies".

```
apt-get -f install
```

This command removes .deb files for packages that are no longer installed on your system. Depending on your installation habits, removing these files from `/var/cache/apt/archives` may regain a significant amount of disk space.

```
apt-get autoclean
```

The same as above, except it removes **all** packages from the package cache. This may not be desirable if you have a slow Internet connection since it will cause you to redownload any packages you need to install a program.

```
apt-get clean
```

- The package cache is in `/var/cache/apt/archives`. The following command will tell you how much space cached packages are consuming.

```
du -sh /var/cache/apt/archives
```

8.5.7 Removal commands

This command removes an installed package, leaving configuration files intact.

```
apt-get remove <package_name>
```

This command completely removes a package and the associated configuration files. Configuration files residing in `~` are not usually affected by this command.

```
apt-get purge <package_name>
```


9 Domain Name System (DNS)

9.1 What is a domain name?

A domain name is a string of text that maps to an alphanumeric IP address, used to access a website from client software. In plain English, a domain name is the text that a user types into a browser window to reach a particular website. For instance, the domain name for Google is 'google.com'.

The actual address of a website is a complex numerical IP address (e.g., 192.0.2.2), but thanks to DNS, users are able to enter human-friendly domain names and be routed to the websites they are looking for. This process is known as a DNS lookup.

The Internet is a giant network of computers connected to each other through a global network. Each computer on this network can communicate with other computers.

To identify them, each computer is assigned an IP address. It is a series of numbers that identify a particular computer on the internet. A typical IP address looks like this 192.0.2.2.

Now an IP address like this is quite difficult to remember. Imagine if you had to use such numbers to visit your favorite websites. Therefore, Domain names were invented to solve this problem. Now, if you want to visit a website, then you don't need to enter a long string of numbers. Instead, you can visit it by typing an easy-to-remember domain name in your browser's address bar. For example, 'google.com'.

9.2 Who manages domain names?

Internet Corporation for Assigned Names and Numbers (ICANN) manages the domain names system. It is a non-profit organization that creates and implements the policies for domain names.

ICANN gives permission to companies called Domain Name Registrars for selling domain names. These domain registrars are allowed to make changes to domain names registry on your behalf.

Domain name registrars can sell domain names, manage its records, renewals, and transfers to other registrars.

As a domain name owner, you are responsible for telling the registrar where to send requests. You are also responsible for renewing your domain registration.

Anyone who wants to create a website can register a domain name with a registrar, and there are currently over 300 million registered domain names

9.3 Different Types of Domain Names

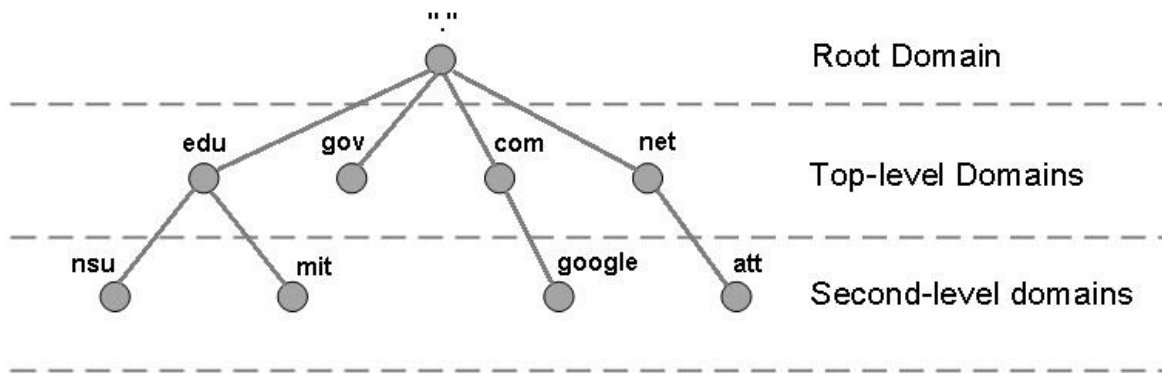
Domain names are available in many different extensions. The most popular one is **.com**. There are many other options like .org, .net, .tv, .info, .io, and more. However, we always recommend using .com domain extension.

Let's take a more detailed look at different types of domain names available.

1. Root Domain

Root Domain is the highest hierarchical level of a site and is separated from the Top-Level Domain by a dot (e.g. rootdomain.com). The term root domain means different things depending on if you're talking about the Internet as a whole or about your website.

Technically, the root domain is the highest hierarchical level of the Internet, even above top-level domains such as .com and .net.



2. Top Level Domain – TLD

A top-level domain (TLD) is the rightmost segment of a domain name, located after the last dot. Also known as domain extensions, TLDs serve to recognize certain elements of a website, such as its purpose, owner or geographical area. For example, a *.edu* top-level domain allows users to immediately identify that site as a higher educational institution.

The ICANN classifies top-level domains into different categories depending on the site's purpose, owner and geographic location.

Generic Top-Level Domains (gTLD)

Generic top-level domains, commonly known as gTLD, are the most popular and familiar types of domain extensions. They are open for registration by anyone and, while the maximum length of top-level domains is 63 characters, most of them are composed of 2-3 letters.

The most common usages of generic top-level domains include:

- **.com** - for commercial sites
- **.org** - for organizations
- **.net** - for networks
- **.info** - for information platforms
- **.biz** - for businesses

Sponsored Top-Level Domains (sTLD)

As the name suggests, sponsored top-level domains are those proposed and supervised by private organizations. These entities can be businesses, government agencies or other types of organized groups, and they have the final word on whether an applicant is eligible to use a specific top-level domain based on predefined community theme concepts.

Among them, some of the most popular sTLDs are:

- **.edu** - for higher educational institutions
- **.gov** - for United States governmental agencies
- **.cat** - for Catalan linguistic and cultural community
- **.museum** - for museum organizations
- **.travel** - for travel industry businesses

Country Code Top-Level Domains (ccTLD)

There are 312 country code top-level domains established for specific countries and territories, identifying them with a two-letter string. These domain extensions have dedicated managers who ensure each ccTLD is operated according to local policies and meets the cultural, linguistic and legal standards of the region.

Some of the most well-known ccTLD include:

- **.us** - for the United States
- **.es** - for Spain
- **.fr** - for France
- **.it** - for Italy
- **.br** - for Brazil
- **.tz** – for Tanzania

Infrastructure Top-Level Domain (ARPA)

This special category contains only one TLD: the Address and Routing Parameter Area (ARPA). The .arpa domain extension is managed directly by the (Internet Assigned Numbers Authority) IANA for the Internet Engineering Task Force (IETF) under the guidance of the Internet Architecture Board (IAB) and is only used for technical infrastructure purposes.

Test top-level domains (tTLD)

Test top-level domains are reserved for documentation purposes and local testing, and cannot be installed into the root zone of the domain name system. According to the IETF, the reason for reserving these specific domain extensions is to reduce the possibility of conflict and confusion.

There are four tTLDs:

- **.example** - for place holding
- **.invalid** - for invalid domain names
- **.localhost** - for usage in local networks
- **.test** - for testing purposes

Unofficial top-level domains

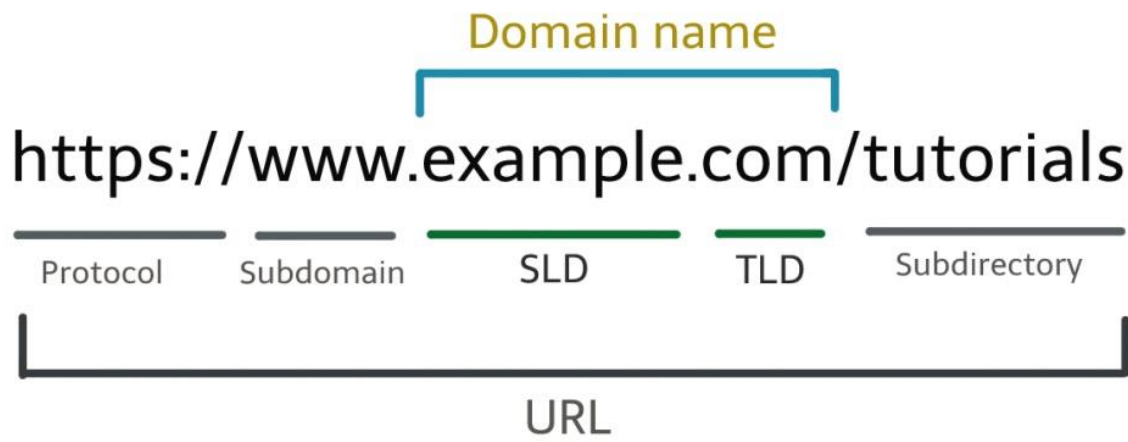
Unofficial top-level domains are those which are not regulated or managed by the ICANN. This type of TLDs are sold and administered by private companies, and as such they aren't in the domain name system and can only be used within a certain network or using a private DNS.

3. Second Level Domain – SLD

Second level domain generally refers to the name that comes before the top-level domain or TLD. For instance, in wpbeginner.com, the wpbeginner is the second-level domain of the .com TLD.

9.4 What is the difference between a domain name and a URL?

A uniform resource locator (URL), sometimes called a web address, contains a site's domain name and other information, including the protocol and the path. For example, in the URL 'https://cloudflare.com/learning/', 'cloudflare.com' is the domain name, while 'https' is the protocol and '/learning/' is the path to a specific page on the website.

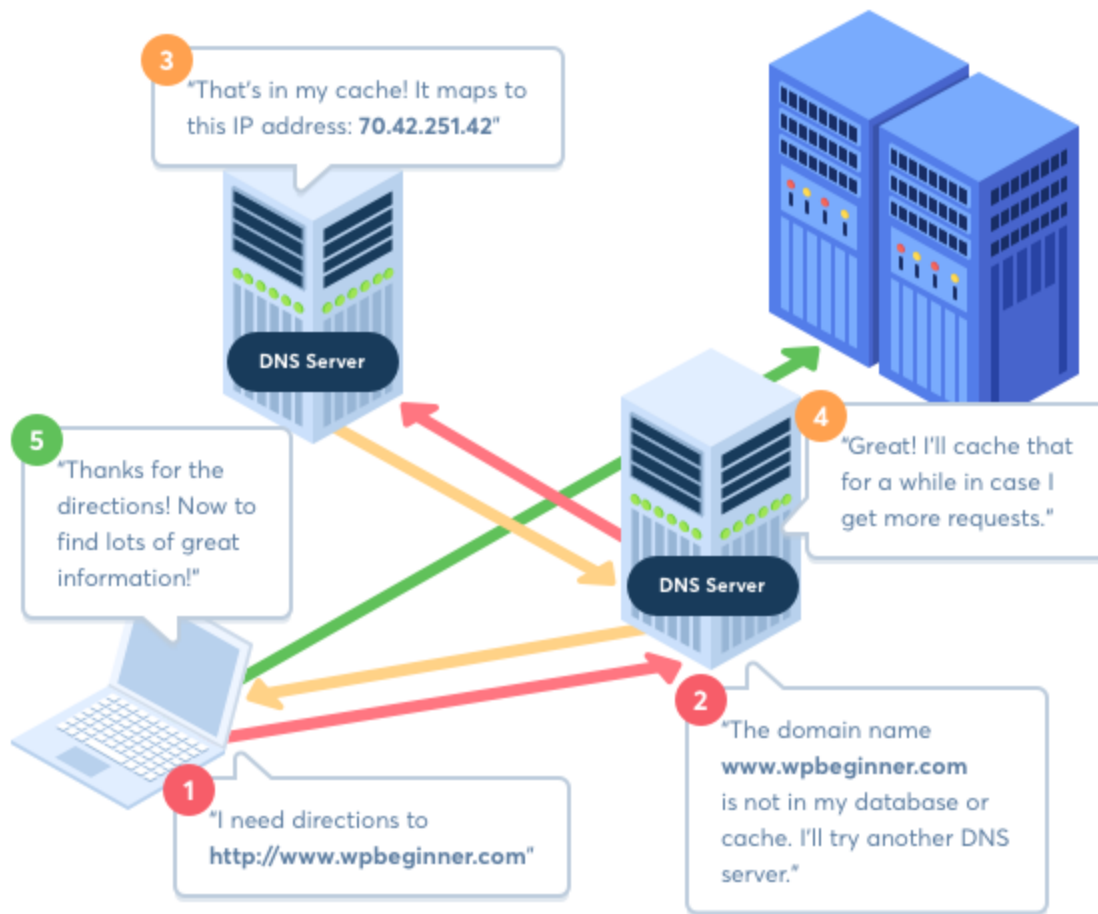


Parts of URL

9.5 How Domain Names Actually Work?

To understand how domain names actually work, we will take a look at what happens when you enter it in your browser.

How Domain Name Works



10 Configuring DNS Server

The following steps should be followed

10.1 Configuring Secondary DNS Server

Step 1: Configure IP Address to the secondary DNS Server

Use the command below to find details of the available adapters, previously configured IP address, subnet mask, and the respective IP information.

```
ip a
```

Or

```
ifconfig -a
```

Output

```
ns1@ns1:/etc/network$ ifconfig -a
enp0s3: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.43.95 netmask 255.255.255.0 broadcast 192.168.43.255
    inet 192.168.1.100 netmask 255.255.255.0 prefixlen 24 scopeid 0x20<link>
    ether 08:00:27:cd:0e:48 txqueuelen 1000 (Ethernet)
    RX packets 291324 bytes 19515185 (19.5 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 490461 bytes 30261543 (30.2 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 905168 bytes 74853662 (74.8 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 905168 bytes 74853662 (74.8 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

The output below indicates that the server has two network adapters, "**enp0s3**" and "**lo**". The

"lo " is a loopback adapter that will not be considered for configurations. Therefore, the **enp0s3**, already configured with IP address **192.168.43.95** and the subnet mask **255.255.255.0**, will be used. As our network topology indicates, this IP address and subnet mask should be updated to **192.168.43.02** with the subnet mask **255.255.255.0**.

Step 2: Configuring IP address 192.168.43.02 and the subnet mask 255.255.255.0.

Netplan is the default network management tool for the latest Ubuntu versions. Configuration files for Netplan are written using YAML and end with the extension **.yaml**. Note: Be careful about spaces in the configuration file, as they are part of the syntax. Without proper indentation, the file won't be read properly. Go to the netplan directory located at **/etc/netplan**. If you do not see any files, you can create one. The name could be anything, but by convention, it should start with a number like **01-** and end with **.yaml**. The number sets the priority if you have more than one configuration file.

Since our netplan directory already contains the file **00-installer-config.yaml**, just update it to reflect configurations of the network topology.

```
ns1@ns1:/etc/netplan$ ls
00-installer-config.yaml
ns1@ns1:/etc/netplan$ sudo nano 00-installer-config.yaml
```

Open the **00-installer-config.yaml** file using nano command and enter configurations.